Read-Write Lock

Read-Write Lock · Cache manifest with many readers · Build with Waqar

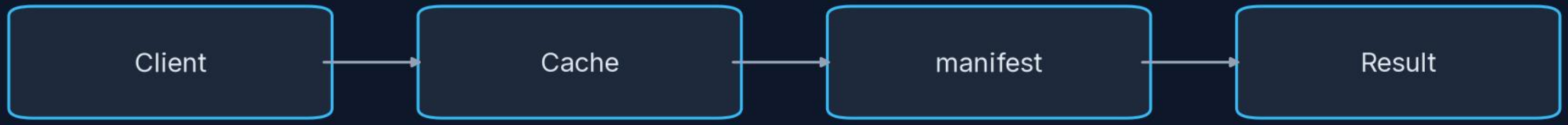
Today's Hook

- Problem: Cache manifest with many readers
- Pattern: Read-Write Lock — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Cache manifest with many readers
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Read-Write Lock: Cache manifest with many readers

Real-World Scenario

- Running example: Cache manifest with many readers
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.concurrency.readwritelock;

import com.buildwithwaqar.patterns.concurrency.readwritelock.CachedValue;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: many threads read a CDN manifest; only one loads it. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Read-Write Lock problem: manifest cache ===");
    var loads = new AtomicInteger();
    var cache = new CachedValue<String>();
    System.out.println(cache.get(() -> { loads.incrementAndGet(); return "manifest-v3"; }));
    System.out.println(cache.get(() -> { loads.incrementAndGet(); return "nope"; }));
    System.out.println("loads=" + loads.get());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.concurrency.readwritelock;

import java.util.concurrent.locks.ReentrantReadWriteLock;
import java.util.function.Supplier;

public final class CachedValue<T> {
    private final ReentrantReadWriteLock lock = new ReentrantReadWriteLock();
    private T value;

    public T get(Supplier<T> loader) {
        lock.readLock().lock();
        try {
            if (value != null) return value;
        } finally {
            lock.readLock().unlock();
        }
        lock.writeLock().lock();
        try {
            if (value == null) value = loader.get();
            return value;
        } finally {
            lock.writeLock().unlock();
        }
```

Solution (2)

```
    }

    public void clear() {
        lock.writeLock().lock();
        try { value = null; } finally { lock.writeLock().unlock(); }
    }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.concurrency.readwritelock;

import com.buildwithwaqar.patterns.concurrency.readwritelock.CachedValue;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: many threads read a CDN manifest; only one loads it. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Read-Write Lock problem: manifest cache ===");
    var loads = new AtomicInteger();
    var cache = new CachedValue<String>();
    System.out.println(cache.get(() -> { loads.incrementAndGet(); return "manifest-v3"; }));
    System.out.println(cache.get(() -> { loads.incrementAndGet(); return "nope"; }));
    System.out.println("loads=" + loads.get());
  }
}
```

Walkthrough Output

```
=== Read-Write Lock problem: manifest cache ===
manifest-v3
manifest-v3
loads=1
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Read-Write Lock.

Related Patterns

- Contrast Read-Write Lock with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Read-Write Lock → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.